Software Development Prompt Worklog

Databricks Student Attrition Risk Application

Feature-001 - Backend Development / US-08

Feature-002 - Briefing Retry and Validated Briefing Storage / US-15

Development method: Claude Code AI agent + GitHub Spec Kit / specification-driven development
Human reviewer: Renny Matis
Purpose: A concise but evidentiary record of the prompts, human decisions, generated development artefacts, implementation outputs, and validation used to create the current software.

> **Human review declaration:** The prompts, AI-generated artefacts, design decisions, analysis findings, and implementation outputs recorded in this worklog were reviewed by **Renny Matis** before being accepted and used in the development process. Stage-specific approval records are retained below.

---

1. How to Read This Worklog

This document deliberately separates **prompt evidence** from **output summaries**.

- The **main worklog** condenses what happened at each development stage.
- Each stage identifies the **prompt ID** that drove the work.
- **Human Review** records that the result was reviewed and approved by **Renny Matis**.
- **Appendix A** preserves the full prompt text used in the workflow so the document remains a genuine prompt worklog rather than only a retrospective description.
- Large AI outputs are summarised rather than reproduced in full; the original source worklog remains the complete raw record.

Development sequence

```
Constitution
-> Repository Investigation
-> Human Design Decisions
-> Specification
-> Clarification
-> Implementation Plan
-> Human Planning Decisions / Scope Correction
-> Tasks
-> Analysis and Remediation
-> Implementation
-> Validation
```

---

2. Shared Project Governance

2.1 Project Constitution

Prompt Used

P01 - Project constitution (`/speckit-constitution`)
Full prompt: **Appendix A - P01**

Output

A project-specific **17-principle constitution** was established for Feature-001, Feature-002 and future application work. It governs specification-driven development, strict scope containment, minimal change, architectural reuse, security/privacy, testing, traceability, protection of team contributions, and human-controlled version control.

Human Review

Constitution.md - human reviewed and approved by Renny Matis.

2.2 Feature-002 Constitution Amendment

Prompt Used

P14 - Amend constitution for Feature-002 scope (`/speckit.constitution`)
Full prompt: **Appendix A - P14**

Output

The constitution was amended from **v1.0.0 to v1.1.0** to correct the ownership boundary:

- **US-14** owns concrete Structured Advisor Briefing validation rules.
- **Feature-002 / US-15** owns the single-retry workflow and governed validated-briefing storage.
- Feature-002 consumes the existing validation boundary rather than duplicating validation logic.
- The original 17 engineering principles remained unchanged.

Human Review

Adjusted Constitution.md - human reviewed and approved by Renny Matis.

---

3. Feature-001 - Backend Development / US-08

3.1 Repository Investigation

Prompt Used

P02 - Repository investigation for Feature-001
Full prompt: **Appendix A - P02**

Output

The repository, Physical Solution Design, ML notebook and synthetic-data notebook were inspected without modifying implementation files. The investigation established the existing architecture and separated findings into **confirmed repository facts**, **implementation context**, and **unresolved product decisions**.

Key findings:

- Streamlit, FastAPI and FastMCP share one `StudentService`.
- Repository/provider behaviour is isolated behind Protocol interfaces.
- Delta Tables provide prediction and student feature data.
- The approved ML model uses 21 features.
- Unity Catalog Volume was the intended briefing store but was not yet implemented.
- The existing deterministic template fallback conflicted with the target failure workflow.

Human Review

Repository findings reviewed by Renny Matis before design decisions and specification work continued.

3.2 Human Product and Behavioural Decisions

Prompt Used

P03 - Human design decisions for specification
Full prompt: **Appendix A - P03**

Output

Human decisions established the behavioural boundary between features and resolved the major product questions. Important decisions included:

- Feature-001 owns the normal backend orchestration workflow.
- The existing ML `attrition_risk_flag` is authoritative.
- All 21 approved ML features are available as briefing context without treating them as individual causal explanations.
- OpenAI is the intended final generative provider.

- Final prompt content, concrete validation and retry/storage remain separately owned by US-12, US-14 and US-15.
- Only validated briefings may enter validated storage.
- Prompts, generated drafts and secrets must not be retained in general logs.

Human Review

Design decisions authored/reviewed and approved by Renny Matis.

3.3 Feature-001 Specification

Prompt Used

P04 - Generate Feature-001 specification (`/speckit-specify`)
Full prompt: **Appendix A - P04**

Output

`spec.md` defined the observable behaviour and acceptance boundaries for **US-08 - Databricks Application Backend**, while deliberately avoiding implementation-level filenames/classes.

Human Review

Specification.md - human reviewed and approved by Renny Matis.

3.4 Feature-001 Clarification

Prompt Used

P05 - Clarify Feature-001 specification (`/speckit-clarify`)
Full prompt: **Appendix A - P05**

Output

Material behavioural ambiguities were resolved. Two important decisions were:

- briefing generation is refused for students not flagged at risk;
- an existing validated briefing is returned by default, with regeneration only on explicit request.

Human Review

Adjusted Specification.md - human reviewed and approved by Renny Matis.

3.5 Feature-001 Implementation Plan

Prompt Used

P06 - Generate Feature-001 implementation plan (`/speckit-plan`)
Full prompt: **Appendix A - P06**

Output

Spec Kit produced:

- `plan.md`
- `research.md`
- `data-model.md`
- `contracts/rest-api.md`
- `contracts/mcp-tools.md`
- `contracts/internal-seams.md`
- `quickstart.md`

The plan reused the existing `StudentService` / Protocol / adapter architecture and defined the minimum implementation structure.

Human Review

Plan.md and supporting planning artefacts - human reviewed and approved by Renny Matis.

3.6 Scope Correction Against Product Backlog

Prompt Used

P07 - Correct Feature-001 scope against Product Backlog
Full prompt: **Appendix A - P07**

Output

Feature-001 was narrowed to **US-08**. Concrete work owned by US-09 through US-26 was removed or retained only as an integration seam. In particular:

- US-12 owns final prompt instructions.
- US-13 owns concrete OpenAI generation.
- US-14 owns concrete validation rules.
- US-15 owns concrete retry and validated-briefing storage.
- US-16 owns final end-to-end integration.

Human Review

Scope correction reviewed and accepted by Renny Matis before task generation.

3.7 Remaining Planning Decisions

Prompt Used

P08 - Approve remaining Feature-001 planning decisions
Full prompt: **Appendix A - P08**

Output

Three planning decisions were explicitly approved:

1. **Get-or-create briefing semantics** - return an existing validated briefing unless regeneration is explicitly requested.
2. **Application-side 21-feature retrieval** - reconstruct the approved feature set from existing Delta Tables without modifying ML code.
3. **HTTP status conventions** - use the approved 404 / 409 / 422 / 502 / 503 mapping.

Human Review

Planning decisions human reviewed and approved by Renny Matis.

3.8 Feature-001 Tasks

Prompt Used

P09 - Generate Feature-001 tasks (`/speckit-tasks`)
Full prompt: **Appendix A - P09**

Output

`tasks.md` translated the approved specification and plan into ordered, file-scoped implementation tasks without redesigning the feature.

Human Review

Tasks.md - human reviewed and approved by Renny Matis.

3.9 Pre-Implementation Analysis and Remediation

Prompt Used

P10 - Analyse Feature-001 before implementation (`/speckit-analyze`)
Full prompt: **Appendix A - P10**

Output

No critical issues were found. Three medium findings were corrected before implementation:

- the 21-feature non-reduction requirement was clarified;
- the superseded silent-template `generate_briefing` path was explicitly removed;
- 21-feature retrieval was kept additive and separate from the existing 11-field student profile.

Human Review

Analysis findings and remediation changes reviewed and accepted by Renny Matis before implementation.

3.10 Feature-001 Implementation

Prompt Used

P11 - Implement Feature-001 (`/speckit-implement`)
Full prompt: **Appendix A - P11**

Core Code Outputs

Feature-001 implemented the backend orchestration and integration seams across:

- `models.py`
- `ports.py`
- `briefing_provider.py`
- `briefing_instructions.py`
- `briefing_validation.py`
- `retry_workflow.py`
- `briefing_store.py`
- `student_service.py`
- `student_repository.py`
- `api.py`
- `mcp_server.py`
- `main.py`
- `config.py`
- minor `ui.py` integration

Architecture Outcome

```
Student / risk retrieval
-> 21-feature context retrieval
-> briefing context construction
-> generation boundary
-> validation boundary
-> retry hand-off boundary
-> validated persistence boundary
-> REST / MCP / application response
```

Feature-001 established the integration seams for later US-12/13/14/15 implementations without prematurely implementing those stories.

Testing Output

Testing included orchestration, seams, store behaviour, REST API, MCP, service and configuration tests.

Validation: 55 tests passed; Ruff clean.

Human Review

Feature-001 implementation and validation output human reviewed and approved by Renny Matis.

4. Feature-002 - Briefing Retry and Validated Briefing Storage / US-15

4.1 Repository Investigation

Prompt Used

P12 - Repository investigation for Feature-002
Full prompt: **Appendix A - P12**

Output

The investigation confirmed that Feature-001 had already created the required architectural seams:

- `RetryWorkflow`
- `RetryNotConfigured`
- `FirstAttemptOutcome`
- `BriefingOutcome`
- `Produced`
- `TerminalFailure`
- `StudentService._hand_off_to_retry`
- `BriefingStore`
- `main.build_service`

The existing suite had **55 passing tests** before Feature-002.

Human Review

Feature-002 repository findings reviewed by Renny Matis before specification work continued.

4.2 Human Feature-002 Design Decisions

Prompt Used

P13 - Human design decisions for Feature-002 specification
Full prompt: **Appendix A - P13**

Output

The approved behavioural decisions established that:

- US-14 owns concrete validation rules; Feature-002 consumes validation results.
- Feature-002 owns both the single-retry workflow and concrete governed Unity Catalog Volume storage required by US-15.
- retry occurs after validation failure or retryable generation/API failure;
- `ConfigurationError` remains immediately non-retryable;
- Attempt 2 is always final;

- validation-failure retry uses actual failed criteria/feedback where available;
- generation-failure retry does not fabricate validation feedback;
- failed attempts are never stored as validated outputs;
- previously stored valid briefings are preserved;
- no new advisor-visible second-attempt label is required.

Human Review

Feature-002 design decisions authored/reviewed and approved by Renny Matis.

4.3 Feature-002 Specification

Prompt Used

P15 - Generate Feature-002 specification (`/speckit.specify`)
Full prompt: **Appendix A - P15**

Output

The specification defined three prioritised behaviour groups:

1. single retry and recovery;
2. terminal failure/no third attempt/preservation of prior valid data;
3. concrete governed Unity Catalog Volume-backed validated storage.

Human Review

Feature-002 Specification.md - human reviewed and approved by Renny Matis.

4.4 Feature-002 Clarification

Prompt Used

P16 - Clarify Feature-002 specification (`/speckit.clarify`)
Full prompt: **Appendix A - P16**

Output

The storage contract was clarified so that the application guarantees retrieval of the **most recent validated briefing per student**. Concrete retention/pruning of older superseded briefings was kept outside the specification.

Human Review

Adjusted Feature-002 Specification.md - human reviewed and approved by Renny Matis.

4.5 Feature-002 Implementation Plan

Prompt Used

P17 - Generate Feature-002 implementation plan (`/speckit.plan`)
Full prompt: **Appendix A - P17**

Output

The plan reused Feature-001's seams and defined:

- concrete `SingleRetryWorkflow`;
- retry-context construction;
- concrete `VolumeBriefingStore`;
- configuration and composition wiring;
- attempt-count behaviour;
- storage format and naming;
- proportionate controlled tests.

Five implementation decisions were approved:

1. configurable `BRIEFING_VOLUME`;
2. unconditional `SingleRetryWorkflow` wiring;
3. minimal retry-feedback wrapper;
4. append-only validated briefing history;
5. one structured JSON file per validated briefing.

Human Review

Feature-002 Plan.md and planning decisions - human reviewed and approved by Renny Matis.

4.6 Feature-002 Tasks

Prompt Used

P18 - Generate Feature-002 tasks (`/speckit.tasks`)
Full prompt: **Appendix A - P18**

Output

`tasks.md` produced **23 ordered implementation tasks (T001-T023)** covering setup, shared foundations, retry success/failure behaviour, Unity Catalog Volume storage, configuration, documentation and validation.

Human Review

Feature-002 Tasks.md - human reviewed and approved by Renny Matis.

4.7 Pre-Implementation Analysis

Prompt Used

P19 - Analyse Feature-002 before implementation (`/speckit.analyze`)
Full prompt: **Appendix A - P19**

Output

The analysis found no critical issues. The principal correction was an additional test case for retry-success followed by storage failure, plus clarification that retry orchestration returns a result while `StudentService` remains responsible for persistence.

Human Review

Task corrections to T006/T007/T012 and the analysis findings were human reviewed and approved by Renny Matis.

4.8 Feature-002 Implementation

Prompt Used

P20 - Implement Feature-002 (`/speckit.implement`)
Full prompt: **Appendix A - P20**

Core Code Outputs

- `retry_workflow.py` - concrete `SingleRetryWorkflow`
- `briefing_store.py` - concrete `VolumeBriefingStore`
- `models.py` - shared `make_validated_briefing(...)`
- `student_service.py` - shared validated-briefing construction integration
- `config.py` - `BRIEFING_VOLUME` and Volume-path validation
- `main.py` - retry/store composition wiring
- `.env.example` / `app.yaml` - deployment configuration placeholder
- `README.md` - Feature-002 configuration/documentation

Behaviour Implemented

```
Attempt 1 failure
-> exactly one retry
-> validation feedback included when it actually exists
-> Attempt 2 generation
-> Attempt 2 validation
-> success: validated briefing, attempt_count = 2
   OR
-> terminal generation/validation failure
-> no third attempt
-> failed briefing not stored
-> previous valid briefing preserved
```

Validated Storage

A governed **Unity Catalog Volume-backed `BriefingStore`** was implemented using one structured JSON file per validated briefing.

Testing Output

Feature-002 added retry, integration, Volume-store and configuration coverage.

Validation:

- **T001-T023 all complete**
- **84 tests passed** (55 Feature-001 baseline -> 84)
- **Ruff clean**
- no Git operations performed by the AI agent

Remaining Deployment/Integration Items

These are not Feature-002 implementation failures:

- supply the real `BRIEFING_VOLUME=/Volumes/<catalog>/<schema>/<volume>` value at deployment;
- ensure the Volume exists and the Databricks App identity has write permission;
- real workspace Files API integration remains to be exercised during later end-to-end integration;
- `build_service` wiring was manually verified rather than directly regression-tested.

Human Review

Feature-002 implementation and validation output human reviewed and approved by Renny Matis.

---

5. Current Backend Architecture

```
Databricks Application Interfaces
    Streamlit UI
    FastAPI REST
    FastMCP
          |
          v
    StudentService
          |
          +--> StudentRepository
          |      -> governed Delta Tables
          |      -> prediction + 21-feature context
          |
          +--> BriefingInstructions
          |
          +--> GenerationProvider
          |
          +--> BriefingValidator
          |
          +--> SingleRetryWorkflow
          |
          +--> BriefingStore
                 -> InMemoryBriefingStore (local/default)
                 -> VolumeBriefingStore (governed deployment)
                        |
```

```
                    v
Unity Catalog Volume
```

The backend is therefore structured so that later concrete US-12, US-13 and US-14 components can be inserted behind existing boundaries rather than requiring a backend redesign.

---

6. Current Development Status

Area	Status
Spec-driven development framework	Complete
Project constitution	v1.1.0
Feature-001 / US-08 backend	Implemented
Feature-001 validation	55 tests passed at completion
Feature-002 / US-15 retry	Implemented
Feature-002 governed storage	Implemented in code
Current automated suite	**84 tests passed**
Ruff	**Clean**
Real `BRIEFING_VOLUME` deployment path	To be configured
Live Volume Files API verification	Deferred to end-to-end integration
US-12 final prompt	Separate backlog work
US-13 concrete OpenAI generation	Separate backlog work
US-14 concrete validation criteria	Separate backlog work
US-16 final end-to-end integration	Future integration work

---

7. Human Review Record

The development process intentionally retained human control rather than treating AI output as self-validating.

Human reviewer: Renny Matis

Recorded review points include:

- constitution approval;
- repository findings and unresolved decision review;
- product/behavioural design decisions;
- specification approval and clarification;

- planning artefacts and technical decisions;
- Product Backlog scope correction;
- task approval;
- pre-implementation analysis and remediation;
- Feature-001 implementation/testing output;
- Feature-002 constitution amendment;
- Feature-002 specification/clarification;
- Feature-002 plan and five planning decisions;
- Feature-002 task corrections after analysis;
- Feature-002 implementation/testing output.

The AI agent was explicitly instructed not to perform commits, pushes, merges, rebases, pull requests or other Git/GitHub write operations unless separately authorised.

Appendix A - Full Prompt Log

The following appendix preserves the **actual prompt text used** in the development workflow. Outputs are summarised in the main body to keep the worklog readable.

P01 - Project constitution

Scope: Shared
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit-constitution

Replace the current constitution. The existing constitution was generated
without sufficient user direction and should be treated as an unapproved draft,
not an adopted project constitution.

Create one constitution governing development of the software application.

The constitution must govern:

Feature-001: Backend functionality supporting the Databricks application.

Feature-002: Structured Advisor Briefing validation and single-retry
functionality.

The constitution should also remain applicable to future software application
features.

Detailed functional requirements, acceptance criteria, workflow behaviour,
and implementation details for Feature-001 and Feature-002 will be defined
through their respective specification, clarification, plan, and task
artifacts. Do not promote feature-specific requirements into constitutional
principles unless they represent permanent cross-feature governance.

For understanding of the existing application and physical solution design,
inspect:

docs/SolutionDesign/T115 Student Risk and Intervention Briefing Documentation.pdf
```

```
Relevant existing data and model implementations may be inspected for
integration context at:

notebooks/Data Generation V2/DBLDatagen Synthetic Data Generation V2.ipynb

notebooks/Machine Learning Model/Machine Learning Model.ipynb

These files provide architectural, data-contract, and integration context.
Their availability for inspection does not grant permission to modify them.
They should remain unmodified unless a later approved implementation plan
explicitly identifies and justifies a required change.

Do not govern development of the existing machine learning model except where
the model imposes a direct constraint on application behaviour, data
interpretation, or integration.

Do not infer additional non-negotiable principles from the repository without
explicitly identifying them to me.

Use the following principles and requirements:

1. Specification-Driven Development — The approved specification is the source
of truth for required behaviour. Behaviour changes must be reflected in the
specification before implementation, with plans, tasks, code, and tests kept
consistent with it.

2. Strict Scope Containment — Development must remain within the files,
modules, and functionality identified by the approved plan and tasks.
Unrelated team code must not be modified. Existing notebooks, model code,
data-generation code, and other team-owned components may be inspected where
necessary to understand interfaces and integration requirements but should
remain unmodified unless the approved plan explicitly requires a change. If
work outside the defined scope is required, the dependency must be identified
and justified before expanding implementation scope.

3. Read Broadly, Write Narrowly — Repository inspection may extend beyond
implementation scope where necessary to understand existing interfaces,
architecture, and dependencies. Permission to inspect repository code does not
imply permission to modify it.

4. Minimal Necessary Change — Implement the simplest change that satisfies the
approved specification. Do not perform unrelated refactoring, cleanup,
optimisation, architectural redesign, or speculative improvements.

5. Reuse and Extend Existing Architecture — Existing application architecture
must be reused and extended wherever reasonably possible. Prefer existing
services, protocols, adapters, repositories, APIs, models, utilities, data
structures, and dependencies before creating new equivalents. Do not create a
parallel backend architecture when the required behaviour can be implemented
through the existing application structure.

6. No Unnecessary Complexity — Do not introduce additional abstractions,
frameworks, dependencies, infrastructure, services, or architectural layers
solely because they represent a theoretically better design. New components
must address an identified requirement or implementation need that cannot
reasonably be satisfied by the existing architecture.

7. Plan-Defined Implementation Structure — Specifications define required
behaviour rather than filenames, classes, or implementation structure. Exact
files, modules, interfaces, and components to create or modify must be
determined during planning after inspection of the existing architecture.
Implementation must then remain within that approved structure unless a
necessary dependency is identified and justified.

8. Application Technology Compatibility — Backend development must remain
compatible with the existing Databricks application environment. Python must
remain the primary backend implementation language, with existing Databricks
technologies such as Delta Tables, Unity Catalog, and Serverless Compute reused
where appropriate.

9. Separation of Responsibilities and Modularity — Preserve the existing
separation between frontend presentation, API/interface layers, backend
orchestration, data access, briefing generation, validation, retry
```

```
orchestration, and persistence. Related functionality should remain modular
enough to be developed and tested independently, but new modules must not be
created where existing components can reasonably support the required
behaviour.

10. Security and Privacy — Student data must use the project's deidentified
student identifier rather than personally identifiable information.
Credentials and API keys must not be hard-coded. Existing Databricks security,
secret-management, and Unity Catalog governance mechanisms must be respected.

11. Input Validation and Explicit Error Handling — Inputs crossing application
boundaries must be appropriately validated. Failures must be handled
explicitly rather than silently ignored, and errors should be propagated in a
form appropriate to the existing application architecture.

12. Proportionate Testing — Testing must be sufficient to verify the approved
acceptance criteria and project testing requirements. Do not create redundant
tests, excessive testing infrastructure, or additional tests solely to
maximise coverage metrics.

13. Human Review of AI-Generated Development Work — AI-generated code, tests,
and important technical documentation must remain subject to human review.
Successful AI generation or passing AI-generated tests alone must not be
treated as proof that an implementation is correct.

14. Documentation and Implementation Traceability — Implementation decisions
and completed work should remain traceable through the specification, plan,
tasks, tests, and relevant project documentation without producing unnecessary
duplicate documentation.

15. Completion Means Specification Satisfaction — Additional functionality,
abstraction, optimisation, or complexity is not evidence of better completion.
A feature is complete when its approved requirements and acceptance criteria
are satisfied within the defined scope.

16. Preserve Team Contributions — Where possible, development should remain
within newly created or explicitly assigned files and avoid unnecessary
changes to files associated with other team members' work. Changes that could
alter another contributor's project evidence should only be made when required
for the approved feature.

17. Human-Controlled Version Control — The AI agent must not commit, push,
merge, rebase, create pull requests, or otherwise modify Git/GitHub history
unless explicitly instructed for that specific action. The agent may inspect
repository status, diffs, branches, and commit history as needed. All commits
and repository-changing Git operations remain under human control by default.

The constitution should remain concise. Do not duplicate detailed feature
requirements, repository inventories, implementation plans, or task-level
instructions within it.

Because the previous constitution was only an unapproved generated draft,
treat this as the initial v1.0.0 constitution rather than a later amendment.

Use 2026-09-02 as the ratification date.
```

P02 - Repository investigation for Feature-001

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
Inspect the repository and relevant project documentation to prepare for
Feature-001 specification development.
```

```
Do not modify any files.

Inspect the existing application architecture and relevant documentation,
including:

docs/SolutionDesign/T115 Student Risk and Intervention Briefing Documentation.pdf

notebooks/Data Generation V2/DBLDatagen Synthetic Data Generation V2.ipynb

notebooks/Machine Learning Model/Machine Learning Model.ipynb

Determine what can already be established from the repository about:

- existing backend architecture;
- student and risk-data retrieval;
- relevant Delta Table sources and available fields;
- existing API operations and response conventions;
- current service, protocol, repository, adapter, and model structures;
- current briefing-generation functionality;
- existing persistence mechanisms;
- existing error behaviour;
- existing testing conventions;
- Databricks integration constraints;
- relevant privacy and security constraints;
- existing boundaries between frontend, API, backend orchestration, data
access, generation, and storage.

For every relevant finding or unresolved question, classify it as:

CONFIRMED FROM REPOSITORY
Directly supported by existing code or project documentation.

REASONABLE IMPLEMENTATION CONTEXT
Existing architecture, conventions, or technical patterns that should probably
be reused during planning but are not themselves feature requirements.

UNRESOLVED PRODUCT DECISION
A behavioural or product decision that cannot be reliably determined from the
repository or project documentation and requires my decision.

Do not invent answers to unresolved product decisions.

Do not propose implementation changes yet.

Do not create or modify specification, plan, task, or implementation files.

Do not commit, push, merge, rebase, create pull requests, or modify Git
history.
```

P03 - Human design decisions for specification

Scope: Feature-001 / Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
I have reviewed the unresolved product decisions identified during repository
investigation. Use the following decisions as authoritative requirements when
creating the Feature-001 and Feature-002 specifications.

FEATURE OWNERSHIP

Use the following behavioural boundary between the two features:

Feature-001 owns the normal backend workflow, including:
```

```
- retrieval of selected student information;
- retrieval of student attrition-risk results;
- retrieval of relevant ML feature values;
- initial prompt construction;
- initial Generative AI generation;
- initiation of briefing validation;
- persistence of validated briefings;
- retrieval of stored validated briefings;
- overall application orchestration and responses.

Feature-002 owns the exceptional retry workflow after the first briefing
attempt does not successfully produce a valid briefing, including:

- capturing validation failure information where validation occurred;
- capturing failed acceptance criteria and validation feedback where available;
- constructing the retry request;
- performing exactly one additional generation attempt;
- validating the second generated briefing;
- terminal failure behaviour after the retry;
- ensuring no third attempt occurs.

This boundary is intended to map the two project features onto the end-to-end
workflow in the Physical Solution Design. Do not duplicate Feature-002 retry
logic inside Feature-001.

STUDENT DATA AND MODEL FEATURES

For Structured Advisor Briefing generation, use the outputs of both the
synthetic data generation process and the Machine Learning Model.

For the selected student_deidentified_hash:

- retrieve the student's approved 21 Machine Learning Model feature values from
the relevant synthetic-data Delta Tables;
- retrieve the student's attrition-risk result from the Machine Learning Model
prediction Delta Table;
- combine these with the other information required by the briefing workflow.

The intention is to make all 21 approved ML model features available as
briefing-generation context rather than limiting the workflow to the existing
11-field PoC snapshot subset.

Some of the 21 model features include demographic attributes. These may be
made available as source context for the briefing workflow, but they must not
be represented as proven causes of an individual student's attrition risk.

The 21 feature values must not be described as per-student SHAP values,
individual causal risk drivers, or evidence that a particular feature caused
the student's risk result. No such per-student explanation currently exists.

Where the Generative AI system interprets relationships between supplied
student information and the risk result, these must be represented as AI
interpretations or possible associations rather than confirmed causation.

If an existing project security, privacy, or platform constraint prevents any
of the 21 approved features from being supplied to the configured Generative
AI service, identify the conflict for human review rather than silently
excluding fields or changing this requirement.

AT-RISK DEFINITION

Use the existing Machine Learning Model threshold as authoritative.

A student is:

- not at risk when the model prediction probability is below 0.50;
- at risk when the model prediction probability is greater than or equal to
0.50.

This corresponds to the existing 50% binary prediction threshold and
attrition_risk_flag.

Where attrition_risk_percentage is represented on a 0–100 scale, this means
```

```
below 50% is not flagged and 50% or above is flagged.

Prefer the existing attrition_risk_flag produced by the Machine Learning Model
rather than introducing a second independently calculated application
threshold.

The percentage/risk score must continue to be treated according to the
documented ML limitations and must not be reinterpreted as a calibrated
probability if the model documentation states otherwise.

GENERATIVE AI PROVIDER

The target Generative AI provider for the final application is the OpenAI API,
as specified in the Physical Solution Design.

The existing DatabricksModelBriefingProvider / Databricks serving-endpoint
implementation is PoC/reference implementation context and must not silently
override the intended OpenAI API target.

Reuse existing provider abstractions where appropriate rather than creating a
parallel backend architecture.

OpenAI credentials must not be hard-coded and must use the appropriate
Databricks secret/configuration mechanism.

If OpenAI integration cannot be implemented within the deployed Databricks App
environment because of a concrete platform constraint, identify that
constraint for human review rather than substituting another provider without
approval.

STRUCTURED BRIEFING INSTRUCTIONS AND ACCEPTANCE CRITERIA

The final:

- Default Structured Briefing Prompt Instructions; and
- Acceptance Criteria for Structured Advisor Briefing

have not yet been finalised or implemented.

Do not invent their final content.

The backend architecture must nevertheless support both so that the completed
documents/criteria can be added later without redesigning the briefing
workflow.

For the interim implementation:

- preserve a replaceable/default briefing-instructions boundary;
- preserve a replaceable validation component/interface;
- use existing safe PoC briefing instructions as temporary behaviour where
appropriate rather than inventing the final instructions;
- allow validation criteria to be supplied or extended later;
- temporarily permit the workflow to operate without the final acceptance
criteria;
- clearly treat any temporary/pass-through validation behaviour as interim
development behaviour rather than the final Structured Advisor Briefing
validation rules.

The retry and validation orchestration itself must still be implemented and
testable using controlled/stub validation outcomes even while the final
acceptance criteria are unavailable.

GENERATION AND RETRY FAILURE BEHAVIOUR

Do not retain the current behaviour where any Generative AI provider exception
is silently converted into a successful deterministic template briefing.

A deterministic fallback must not masquerade as a successfully generated and
validated Structured Advisor Briefing.

The single retry workflow must activate when either:

1. Attempt 1 produces a briefing that fails validation; or
```

```
2. Attempt 1 fails to generate a briefing because the Generative AI provider/API
fails.

For a validation failure:

- record failed acceptance criteria where available;
- produce/use validation feedback;
- retry using the original source inputs and instructions plus failed criteria
and validation feedback.

For a Generative AI provider/API failure before a briefing is generated:

- perform the one permitted retry using the same original student data, risk
result, model features and briefing instructions;
- do not fabricate failed acceptance criteria or validation feedback when no
briefing existed to validate.

Attempt 2 is the final generation attempt.

If Attempt 2 fails because of either generation failure or validation failure:

- terminate the workflow;
- perform no third attempt;
- return an application-visible error;
- do not store a briefing as a validated output.

PERSISTENCE

Validated Structured Advisor Briefings must be persisted in a governed
Databricks Unity Catalog Volume.

Only briefings that have passed the applicable validation stage may be stored
as validated briefing artefacts.

Failed drafts must not be permanently stored as validated briefings.

Workflow metadata persistence is OPTIONAL rather than a Must Have requirement.

If useful metadata can be persisted with minimal implementation complexity
using the existing architecture, include proportionate metadata such as:

- student_deidentified_hash;
- generation timestamp;
- attempt count;
- success/failure status;
- relevant validation result identifiers;
- failure category where appropriate.

Do not introduce a new Delta Table, substantial infrastructure, or significant
additional complexity solely to persist workflow metadata. If metadata would
require substantial additional implementation, omit it.

Exact Unity Catalog Volume path, file format, naming/keying strategy and other
low-level persistence decisions should be determined during /speckit-plan
after inspection of the existing architecture.

STORED BRIEFING RETRIEVAL

Retrieval is a required Feature-001 capability.

Validated briefings stored in the Unity Catalog Volume must be capable of
being retrieved and displayed through the Databricks application for academic
advisors.

Only validated/finalised briefings should be exposed through this stored
briefing retrieval workflow.

The exact API routes, methods and implementation structure should be determined
during /speckit-plan rather than prescribed in the specification.

LOGGING AND PRIVACY

Do not log or otherwise retain full Generative AI prompts or generated
```

```
briefing text as general application logs.

Validated briefing content may be persisted only through the designated
validated-briefing storage mechanism in Unity Catalog.

Failed generated briefing content should not be permanently retained.

Only minimal workflow/error metadata necessary for operation, testing,
traceability or safe troubleshooting should be retained.

Do not log credentials, API keys, tokens, or other secrets.

DEFER IMPLEMENTATION-LEVEL UNRESOLVED ITEMS

Remaining questions that concern exact modules/files, synchronous versus
asynchronous implementation, endpoint paths/HTTP verbs, pagination details,
Unity Catalog Volume path/file format, protocol/class design, deployment
catalog configuration, and other low-level implementation choices should not
be treated as missing product requirements at this stage.

Resolve those during /speckit-plan by inspecting and extending the existing
architecture with the minimum necessary change.

If one of those apparently technical decisions would materially change
user-visible behaviour or project scope, flag it for human approval rather
than silently deciding it.
```

P04 - Generate Feature-001 specification

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit-specify

Create the Feature-001 specification using:

- the project constitution;
- the repository findings from the previous investigation prompt;
- the product and behavioural decisions I provided in response to the unresolved questions;
- the referenced Physical Solution Design documentation;
- the existing source code as implementation context only.

Treat my confirmed answers to the unresolved product decisions as authoritative.

Do not reopen decisions that have already been resolved unless there is a direct conflict with the constitution
or project documentation. If a conflict exists, identify it explicitly rather than silently choosing one source.

Use repository findings to inform the specification, but do not turn low-level implementation details into
specification requirements.

Define WHAT Feature-001 must do and its observable behaviour, acceptance criteria, boundaries, dependencies, and
failure behaviour.

Do not prescribe exact filenames, classes, modules, interfaces, or implementation structure. Those belong in /
speckit-plan.

Do not invent unresolved Structured Advisor Briefing instructions or acceptance criteria. Preserve the agreed
temporary/extensible behaviour so these can be added later.

Do not modify implementation code, commit, push, merge, create pull requests, or modify Git history.
```

P05 - Clarify Feature-001 specification

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit-clarify

Review the Feature-001 specification and identify only remaining material
ambiguities that cannot already be resolved from:

- the approved specification;
- the project constitution;
- confirmed repository findings;
- referenced project documentation;
- my confirmed answers to the previously identified unresolved product decisions.

Do not reopen decisions that have already been explicitly resolved.

Do not ask about low-level implementation or architectural choices that belong
in /speckit-plan.

Ask only questions requiring a genuine behavioural, product,
acceptance-criteria, or workflow decision from me.

Do not invent answers.

If no material ambiguity remains, state that clarification is complete rather
than manufacturing additional questions.

Integrate my confirmed answers back into the specification.
```

P06 - Generate Feature-001 implementation plan

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit-plan

Create the implementation plan for Feature-001 using the approved
Feature-001 specification and project constitution.

Inspect the existing repository before determining implementation structure.

Use the confirmed repository findings and clarified specification as context.

Reuse and extend the existing application architecture wherever reasonably
possible. Determine the minimum implementation necessary to satisfy the
specification.

Specifications define required behaviour; determine exact files, modules,
interfaces and components during this planning stage.

Prefer existing services, protocols, adapters, repositories, API structures,
models and utilities before creating new equivalents.

Do not redesign the application or create a parallel backend architecture.

Clearly identify:
```

```
- existing files that will be modified;
- existing files that will remain read-only;
- any new files that are genuinely required;
- how Feature-001 integrates with Feature-002;
- data retrieval and Delta Table integration;
- OpenAI provider integration;
- validation integration;
- Unity Catalog Volume persistence and retrieval;
- error handling;
- proportionate testing required to demonstrate the specification.

Treat implementation-level choices that were deliberately deferred from the
specification as planning decisions.

If a technical decision would materially change approved behaviour or feature
scope, flag it for human approval rather than silently changing the
specification.

Do not modify implementation code during planning.

Do not commit, push, merge, rebase, create pull requests, or modify Git
history.
```

P07 - Correct Feature-001 scope against Product Backlog

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
Before generating Feature-001 tasks, perform a scope correction against the
Product Backlog.

Feature-001 corresponds to:

US-08 — Databricks Application Backend

"As an Academic advisor, I want to use an application backend that retrieves
risk data and coordinates briefing requests, so that dashboard actions and
the student briefing workflow operate reliably."

Review the Feature-001 specification, plan.md, research.md, data-model.md,
contracts, and quickstart against the remaining Product Backlog user stories.

The following backlog boundaries are authoritative:

US-09 — Databricks Dashboard Frontend
Owns implementation of the advisor-facing dashboard/frontend.

US-10 — At-Risk Student Display
Owns advisor-facing display of at-risk students. Feature-001 may provide the
backend risk-data retrieval required by this story.

US-11 — Student Selection and Briefing Request Functionality
Owns the advisor-facing student-selection and request interaction.
Feature-001 may expose the backend capability that receives that request.

US-12 — Default Structured Briefing Prompt
Owns the final reusable Structured Advisor Briefing prompt, instructions,
sections, language guidance, and acceptance-criteria content.
Feature-001 may provide only the integration seam or temporary placeholder
needed to consume these instructions later.
```

```
US-13 — Generative AI Briefing Generation
Owns the concrete Generative AI/OpenAI API implementation that generates a
draft Structured Advisor Briefing.
Feature-001 may invoke a BriefingProvider or equivalent abstraction and handle
its result/failure, but must not implement the concrete OpenAI integration.

US-14 — Structured Advisor Briefing Validation
Owns implementation of the actual Structured Advisor Briefing acceptance-
criteria validation behaviour.
Feature-001 may define/use the validator seam and temporary test/pass-through
behaviour but must not implement acceptance criteria that have not yet been
defined.

US-15 — Briefing Retry and Validated Briefing Storage
Owns the concrete one-retry behaviour and validated-briefing storage behaviour.
This corresponds to Feature-002 for retry behaviour.
Feature-001 may define the required retry and persistence integration seams,
but must not prematurely implement the later user story.

US-16 — End-to-End Component Integration
Owns final integration of the ML, application, Generative AI, validation,
retry and storage components.
Feature-001 should be compatible with that integration but should not expand
its scope simply to complete US-16 early.

US-17 and US-18 own broader application/dashboard and complete briefing-
workflow testing. Feature-001 should contain only proportionate tests needed
to verify its own acceptance criteria and integration seams.

US-19 through US-23 are later refinement, defect-resolution, final delivery,
documentation and handover work and must not be implemented as part of
Feature-001.

US-24, US-25 and US-26 are separate solution-design, synthetic-data validation,
and UI/UX design work and are outside Feature-001.

For Feature-001, retain as in-scope:

- retrieval of existing student data from governed Delta Tables;
- retrieval of the 21 approved ML feature values for the selected student;
- retrieval of the student's existing prediction/risk result;
- high-risk backend retrieval using the approved model risk flag;
- backend request processing and orchestration;
- service/API/MCP integration required for the backend;
- contracts/interfaces needed to integrate later briefing-generation,
validation, retry and persistence capabilities;
- explicit handling of dependency results/failures;
- proportionate Feature-001 tests.

Do not modify the existing ML or synthetic-data implementations.

Audit the current Feature-001 specification and planning artifacts for
requirements or implementation work that actually belongs to US-09 through
US-26.

For each overlap, classify it as:

1. KEEP IN FEATURE-001
Required backend responsibility.

2. KEEP AS INTEGRATION SEAM ONLY
Feature-001 needs the interface/contract but the concrete implementation belongs
to another backlog story.

3. REMOVE/DEFER
Concrete functionality belongs to another user story and is not required to
satisfy US-08.

In particular, reassess:

- briefing_instructions.py;
- briefing_validation.py;
- briefing_store.py;
```

```
- retry_workflow.py;
- concrete OpenAI integration;
- concrete Unity Catalog Volume writing;
- retry implementation;
- final briefing validation;
- regeneration behaviour;
- broad end-to-end workflow tests.

If correcting the scope requires changing behavioural requirements currently
present in spec.md, identify those changes first because the specification is
the source of truth. Do not silently make the plan contradict the approved
specification.

Update the Feature-001 specification and planning artifacts only as necessary
to restore the correct backlog boundaries.

Do not implement code yet.
Do not generate tasks until the scope correction is complete.
Do not modify unrelated user-story functionality.
Do not perform Git operations.
```

P08 - Approve remaining Feature-001 planning decisions

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
I approve the three remaining Feature-001 planning decisions as follows.

1. POST briefing semantics

Choose Option A: get-or-create behaviour.

If a validated briefing already exists for the selected student, the normal
briefing request should return the existing validated briefing without
triggering another generation request.

A fresh briefing should only occur through an explicit regeneration request.

Feature-001 owns only the backend orchestration/routing behaviour. Concrete
generation, validation, retry, and persistent storage remain owned by their
respective later backlog stories.

2. Retrieval of the 21 ML features

Choose Option A: application-side retrieval/join.

Feature-001 should retrieve and assemble the selected student's 21 approved ML
feature values from the existing synthetic-data Delta Tables using the existing
identifiers and relationships, and combine these with the student's prediction
result from the existing ML prediction Delta Table.

Do not modify the Machine Learning Model or synthetic-data-generation
implementations and do not require the ML team to create an additional
projection table.

Record this explicitly as a design decision and technical risk:

The application will reproduce the joins required to reconstruct the approved
21-feature input set rather than consuming a dedicated ML-generated feature
projection table. This may create maintenance risk if the ML feature set,
source-table relationships, or transformation logic changes later, because the
application-side retrieval logic may also need to be updated.

Mitigate this by:
```

```
- treating the approved 21-feature contract as authoritative;
- keeping feature-retrieval/join logic isolated within the data-access layer;
- avoiding duplication of ML transformation logic beyond what is necessary to
retrieve the existing feature values;
- allowing a future canonical feature-projection table to replace the
application-side join without changing Feature-001 orchestration.

This risk is accepted because modifying the existing ML implementation is
outside Feature-001 scope.

3. HTTP error/status conventions

Approve the proposed status-code approach where consistent with the existing
FastAPI application conventions:

- 404 — requested student/resource does not exist;
- 409 — a valid request conflicts with application state, including requesting
briefing generation for a student who is not flagged at risk;
- 422 — request/input validation, preferably using existing FastAPI/Pydantic
behaviour;
- 502 — an invoked downstream briefing-generation dependency fails;
- 503 — required Databricks/data/backend infrastructure is unavailable.

Do not introduce additional status codes unless required by the approved
Feature-001 specification.

Keep external error responses safe and concise while preserving appropriate
internal exception handling.

These three planning decisions are approved for Feature-001.

Update plan.md, research.md, contracts, and any other affected planning
artefacts as necessary to record these decisions before generating
/speckit-tasks.

Do not expand Feature-001 scope while applying these decisions.
```

P09 - Generate Feature-001 tasks

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit-tasks

Generate the implementation tasks for Feature-001 from the approved,
scope-corrected specification and final planning artifacts.

Feature-001 is strictly bounded to US-08 — Databricks Application Backend.

Treat the constitution, spec.md, plan.md, research.md, data-model.md, contracts,
quickstart.md, and approved planning decisions as authoritative.

Generate only tasks required to implement the approved Feature-001 plan.

Preserve the exact scope, files, interfaces, integration seams, and backlog
boundaries already defined in the plan.

Do not implement functionality deferred to later user stories.

Order tasks by dependency, reference the planned files/modules, and associate
proportionate tests with the behaviour they verify.

Do not introduce new architecture, dependencies, refactoring, cleanup,
optimisation, or functionality beyond the approved plan.
```

```
Do not modify the Machine Learning Model or synthetic-data-generation
implementations.

If the task-generation process reveals a conflict between the approved
artifacts, stop and identify it rather than silently resolving it.

Do not implement code or perform Git operations.
```

P10 - Analyse Feature-001 before implementation

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit-analyze

Analyze Feature-001 for consistency across the approved constitution,
specification, clarification decisions, final plan, supporting planning
artifacts, and generated tasks.

Feature-001 is strictly bounded to US-08 — Databricks Application Backend.

Check for:

- requirements without corresponding tasks;
- tasks without corresponding approved requirements;
- contradictions between spec, plan, contracts, and tasks;
- scope leakage into later backlog user stories;
- missing acceptance-criteria coverage;
- unnecessary implementation complexity;
- constitution violations.

Do not modify implementation code.

If issues are found, identify the exact artifact and issue that should be
corrected before implementation.

Do not perform Git operations.
```

P11 - Implement Feature-001

Scope: Feature-001
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit-implement

Implement Feature-001 using the approved constitution, specification,
implementation plan, and tasks.

Treat the approved tasks and the exact files identified by the approved plan
as the implementation boundary.

Requirements:

- Follow the approved specification, plan, and tasks exactly.
- Modify only files explicitly identified by the approved plan/tasks as files
```

```
to be modified or created.
- Files identified as read-only, reference-only, outside Feature-001 scope, or
associated with other team members must not be modified.
- Existing notebooks, machine-learning code, synthetic-data-generation code,
evidence artefacts, documentation owned by other contributors, and unrelated
application components may be inspected where necessary but must remain
unchanged unless an approved task explicitly authorises modification.
- Do not modify a neighbouring or dependent file merely because doing so would
make implementation easier, cleaner, or more architecturally convenient.
- If implementation requires changing any file not already authorised by the
approved plan/tasks, STOP before making that change and report:
1. the file;
2. why the change appears necessary;
3. the requirement or dependency causing it;
4. the minimum proposed change;
5. whether existing team functionality could be affected.
Wait for human approval before expanding implementation scope.
- Do not overwrite, replace, restructure, rename, or delete existing
team-created functionality unless an approved task explicitly requires it.
- Preserve all existing application behaviour outside Feature-001 scope.
- Reuse and extend the existing architecture only within the approved
implementation structure.
- Do not create parallel implementations when an approved existing component
is intended to be extended.
- Do not reinterpret, expand, or add requirements during implementation.
- Do not perform unrelated refactoring, cleanup, optimisation, architectural
redesign, dependency upgrades, formatting changes, or speculative
improvements.
- Avoid repository-wide formatting or automated fixes that could modify files
outside the approved scope.
- Implement only the minimum code necessary to satisfy each approved task.
- Complete the associated tests defined by the approved tasks and plan.
- Tests may be added or modified only in the files authorised by the plan/tasks.
- Run the specified Ruff and pytest validation gates, but do not use automatic
fix options that could alter unrelated files.
- Do not alter the approved specification, plan, or tasks simply to make them
match the implementation. Surface inconsistencies for human review.
- If a requirement conflict, missing dependency, architectural conflict, or
scope expansion is discovered, stop at that boundary and report it rather
than silently resolving it through additional changes.
- Do not commit, push, merge, rebase, create pull requests, switch branches,
amend commits, or otherwise modify Git/GitHub history.

Before making changes:
1. Inspect git status and the approved task/file scope.
2. Identify the files expected to be modified or created.
3. Confirm that no unrelated or team-owned files need modification.

During implementation:
- Work through tasks in their approved dependency order.
- Keep changes narrowly scoped to the current task.
- Do not modify files outside the approved implementation boundary.

After implementation:
- Report all files created or modified.
- Report which tasks were completed.
- Report tests and validation commands run and their results.
- Report any files considered for modification but deliberately left unchanged
because they were outside scope.
- Report any unresolved issues requiring human review.

Do not perform Git/GitHub write operations.
```

P12 - Repository investigation for Feature-002

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
Inspect the repository and relevant project documentation to prepare for
Feature-002 Retry Workflow specification development.

Do not modify any files.

Read and comply with the existing constitution.md first.

Inspect the existing application architecture and relevant documentation,
including:

- constitution.md
- docs/SolutionDesign/T115 Student Risk and Intervention Briefing Documentation.pdf
- current Databricks application/backend source
- existing briefing-generation functionality
- existing validation functionality or interfaces
- existing persistence/storage functionality or interfaces
- Feature-001 specification, clarification, plan, tasks, and implementation where relevant
- Product Backlog US-14 and US-15
- Release Roadmap
- existing application tests

You may inspect other repository files where necessary to understand interfaces,
architecture, dependencies, and existing conventions.

Permission to inspect does not grant permission to modify.

Feature-002 concerns the exceptional retry workflow after the first Structured
Advisor Briefing attempt does not successfully produce a valid briefing.

Use the Physical Solution Design as the primary contextual source for the intended
retry workflow.

Determine what can already be established from the repository about:

- the current end-to-end briefing workflow;
- where initial briefing generation occurs;
- current Generative AI provider interfaces and failure behaviour;
- existing validation interfaces and validation-result structure;
- whether failed acceptance criteria and Validation Feedback are currently available;
- existing retry behaviour, if any;
- how the current architecture distinguishes generation failure from validation failure;
- how retry prompt/request construction could integrate with the existing generation boundary;
- where retry orchestration naturally fits within the current service architecture;
- how exactly one retry can be enforced;
- current terminal error behaviour;
- existing validated-briefing persistence behaviour;
- whether invalid or failed drafts are currently stored;
- current REST, MCP, frontend, service, generation, validation, and persistence boundaries;
- existing Pydantic models, protocols, services, adapters, and dependency injection;
- existing testing conventions for controlled success/failure outcomes;
- relevant Databricks, OpenAI, Unity Catalog, security, privacy, and logging constraints;
- the integration boundary between Feature-001 and Feature-002.

Explicitly compare the current repository with the retry workflow described in
the Physical Solution Design and US-15, including:

- first briefing attempt;
- validation;
- failed acceptance criteria and Validation Feedback;
- revised retry request;
- exactly one additional generation attempt;
- second validation;
- successful continuation when the retry passes;
- terminal error when the retry fails;
- no third attempt;
- no invalid briefing stored as a validated output.

Also investigate whether the current behaviour of silently replacing a failed
Generative AI request with a deterministic/template briefing would conflict with
```

```
the intended Feature-002 workflow.

For every relevant finding or unresolved question, classify it as:

CONFIRMED FROM REPOSITORY / APPROVED DOCUMENTATION
Directly supported by existing code or approved project documentation.

REASONABLE IMPLEMENTATION CONTEXT
Existing architecture, conventions, interfaces, or technical patterns that should
probably be reused during planning but are not themselves feature requirements.

UNRESOLVED PRODUCT DECISION
A behavioural or scope decision that cannot be reliably determined from the
repository or approved documentation and requires my decision.

DEFERRED IMPLEMENTATION DECISION
A low-level technical decision that should be resolved during /speckit-plan rather
than during specification.

Do not invent answers to unresolved product decisions.

Do not propose implementation changes yet.
```

P13 - Human design decisions for Feature-002 specification

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
I have reviewed the unresolved product decisions identified during the
Feature-002 repository investigation.

Use the following decisions as authoritative requirements when creating the
Feature-002 specification.

Do not reopen decisions already established by Feature-001 unless there is a
direct conflict with these decisions.

FEATURE-002 SCOPE

Feature-002 owns the exceptional single-retry workflow after the first
Structured Advisor Briefing attempt does not successfully produce a valid
briefing.

US-14 owns the concrete Structured Advisor Briefing validation rules.

Feature-002 consumes the existing validation boundary and validation result,
including failed acceptance criteria and Validation Feedback where available,
and invokes validation again on the second generated briefing.

Do not duplicate the US-14 validator implementation inside Feature-002.

US-15 STORAGE BOUNDARY

Preserve the persistence architecture established by Feature-001.

Feature-002 should return a successfully validated retry result through the
existing RetryWorkflow boundary and allow the existing StudentService /
BriefingStore workflow to persist it.

Do not duplicate persistence orchestration inside the retry workflow.

Where US-15 requires the remaining concrete Unity Catalog Volume storage
```

```
implementation, implement it through the existing BriefingStore abstraction
rather than introducing a parallel storage architecture.

RETRY TRIGGERS

The single retry workflow applies when Attempt 1:

1. generates a briefing that fails validation; or
2. fails because of a retryable Generative AI provider/API error before a draft
briefing is produced.

Preserve the existing ConfigurationError behaviour where the repository already
treats an unconfigured provider as immediately non-retryable.

VALIDATION FAILURE RETRY

When Attempt 1 fails validation:

- preserve the original briefing-generation context;
- include failed acceptance criteria where available;
- include Validation Feedback where available;
- construct a revised retry request;
- perform exactly one additional generation attempt.

Do not invent failed criteria or feedback that were not produced by validation.

GENERATION FAILURE RETRY

When Attempt 1 fails before producing a draft:

- perform the one permitted retry using the original generation context;
- do not fabricate validation feedback or failed acceptance criteria.

ATTEMPT 2

Attempt 2 is always the final generation attempt.

If Attempt 2 fails during generation:
- terminate with generation failure.

If Attempt 2 generates a briefing but it fails validation:
- terminate with validation failure.

If Attempt 2 passes validation:
- return the validated briefing through the existing successful retry outcome;
- use attempt_count = 2.

No third generation attempt is permitted.

TERMINAL FAILURE

After Attempt 2 fails:

- display/return the existing application-visible briefing failure;
- perform no third attempt;
- do not substitute a deterministic/template briefing;
- do not store the failed briefing as a validated output;
- retain any previously stored valid briefing.

RETRY PROMPT

For validation failures, the retry request must use the original briefing
context together with failed acceptance criteria and Validation Feedback where
available.

Do not define the exact retry prompt wording in the specification.

The exact prompt-construction mechanism belongs in /speckit-plan.

LEGACY PROVIDERS

Leave the currently unwired TemplateBriefingProvider and
DatabricksModelBriefingProvider unchanged unless later planning identifies a
```

```
specific requirement to modify them.

Do not expand Feature-002 scope for unrelated cleanup.

ADVISOR RETRY VISIBILITY

Do not introduce a new requirement to visibly label a successful briefing as a
second attempt.

The existing attempt_count is sufficient for workflow traceability unless a
separate frontend requirement is approved.

DEPENDENCY ON US-13 AND US-14

Feature-002 must be independently testable using controlled GenerationProvider
and BriefingValidator outcomes.

It does not need to wait for the final US-13 OpenAI implementation or final
US-14 acceptance criteria before the retry orchestration itself can be
implemented and tested.

The final implementations must be able to plug into the existing boundaries
without redesigning Feature-002.

DEFER IMPLEMENTATION DETAILS

Do not treat exact:
- module/class names;
- constructor wiring;
- prompt-builder implementation;
- context-copy mechanics;
- Volume path or file format;
- configuration names;
- test file placement;
- logging helper details

as unresolved specification questions.

Resolve those during /speckit-plan using the existing architecture and the
minimum necessary change.
```

P14 - Amend constitution for Feature-002 scope

Scope: Shared / Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit.constitution

Amend the existing project constitution to resolve the Feature-002 scope conflict
identified during repository investigation.

Update only the "Scope and Applicability" section, together with the required
version metadata and Sync Impact Report.

FEATURE-002 SCOPE

The current constitution describes Feature-002 as owning "Structured Advisor
Briefing validation and single-retry functionality."

Amend this so that Feature-002 instead owns Product Backlog US-15 in full:

- the exceptional single-retry workflow after the first Structured Advisor
Briefing attempt does not successfully produce a valid briefing; and
- the governed storage of validated Structured Advisor Briefings, implemented
```

```
through the existing BriefingStore architecture and backed by Unity Catalog
Volume storage.

Clarify that the concrete Structured Advisor Briefing validation rules and
acceptance criteria are owned by US-14.

Feature-002 consumes the existing validation boundary and its results as part of
the retry workflow, but does not define or duplicate the US-14 validation logic.

RATIONALE

Repository investigation identified a scope conflict between the current
constitution and the approved Product Backlog / Feature-001 architecture.

The intended decomposition is:

- US-14 owns concrete Structured Advisor Briefing validation;
- Feature-002 / US-15 owns the single-retry workflow and validated briefing
storage.

Feature-001 has already established the architectural seams that Feature-002
should reuse and extend rather than replace.

This amendment should align the constitution with that feature ownership without
changing the project's existing development principles.

PRESERVE EXISTING PRINCIPLES

Do not add, remove, rewrite, or reinterpret any constitutional principle.

Do not introduce Feature-002 behavioural or implementation details such as:

- exact retry triggers or terminal failure behaviour;
- retry prompt construction;
- module or class names;
- dependency-injection wiring;
- Unity Catalog Volume paths or file formats;
- configuration names;
- API implementation details;
- test structure.

Those belong in the Feature-002 specification and subsequent /speckit.plan.

VERSIONING

Bump the constitution from version 1.0.0 to 1.1.0.

Update:
- Version;
- Last Amended using the current date;
- Sync Impact Report.

The Sync Impact Report should note that:
- Feature-002 is now defined as US-15 single retry + governed validated briefing
storage;
- US-14 retains ownership of concrete validation rules;
- Scope and Applicability is the only materially changed section;
- no constitutional principles were changed.

Do not modify unrelated sections.

Do not perform any Git or GitHub write operations.
```

P15 - Generate Feature-002 specification

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit.specify

Create the Feature-002 specification for Product Backlog US-15:
Briefing Retry and Validated Briefing Storage.

Use:
- the amended project constitution v1.1.0;
- the completed Feature-002 repository investigation and my confirmed design
decisions;
- the merged Feature-001 specification, plan, research, data model, contracts,
quickstart and tasks;
- Product Backlog US-15, Release Roadmap and Physical Solution Design;
- the existing source code as implementation context only.

Treat my confirmed answers to the unresolved Feature-002 product decisions as
authoritative. Do not reopen them unless they directly conflict with the
constitution or authoritative project documentation.

CONFIRMED FEATURE-002 DECISIONS

1. Feature-002 completes US-15 in full:
- the exceptional single-retry workflow; and
- the concrete governed Unity Catalog Volume-backed implementation of the
existing BriefingStore abstraction.

2. US-14 owns the concrete Structured Advisor Briefing validation rules and
acceptance criteria. Feature-002 consumes the existing validation boundary
and ValidationOutcome rather than defining or duplicating validation logic.

3. Exactly one retry is allowed after either:
- Attempt 1 validation failure; or
- a retryable GenerationProvider failure.
Any GenerationProvider failure other than ConfigurationError is retryable.
ConfigurationError remains immediately non-retryable.

4. For a validation failure, the retry uses the original briefing-generation
context together with failed acceptance criteria and Validation Feedback where
available. Do not invent missing criteria or feedback.

For a generation failure before a draft exists, retry using the original
context without fabricating validation feedback.

5. Attempt 2 is always final:
- generation failure -> terminal generation failure;
- generated briefing fails validation -> terminal validation failure;
- validation passes -> successful validated briefing with attempt_count = 2.
No third generation attempt is permitted.

6. After terminal failure:
- return the existing application-visible failure;
- do not substitute a deterministic/template briefing;
- do not store the failed briefing as validated output;
- preserve any previously stored valid briefing.

7. Preserve Feature-001 persistence orchestration. Feature-002 provides the
concrete Unity Catalog Volume-backed BriefingStore, but RetryWorkflow itself
does not duplicate persistence responsibility.

8. Do not require a new advisor-visible indication that a successful briefing
came from Attempt 2. Existing attempt_count behaviour is sufficient.

9. Leave the currently unwired TemplateBriefingProvider and
DatabricksModelBriefingProvider unchanged. Do not expand Feature-002 for
unrelated cleanup.

10. Feature-002 must be independently testable using controlled/stub
GenerationProvider and BriefingValidator outcomes. Final US-13 generation
and US-14 validation implementations are not hard prerequisites for
specifying and testing the retry workflow.
```

```
Do not invent unresolved US-13 generation details, US-14 validation criteria,
or exact retry-prompt wording.

Preserve and extend the architecture and seams established by Feature-001 rather
than redesigning them.

Define WHAT Feature-002 must accomplish: observable behaviour, acceptance
criteria, boundaries, dependencies and failure behaviour.

Do not prescribe exact filenames, classes, modules, dependency-injection wiring,
context-copy mechanics, retry-prompt implementation, Unity Catalog Volume
paths/file formats, configuration names, logging helpers, test-file placement,
or sync/async choices. Resolve those during /speckit.plan.

Do not modify implementation code or perform any Git/GitHub write operations.
```

P16 - Clarify Feature-002 specification

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit.clarify

Review the approved Feature-002 specification and identify only remaining
material ambiguities that cannot already be resolved from:

- the approved Feature-002 specification;
- the amended project constitution v1.1.0;
- the completed Feature-002 repository investigation;
- Product Backlog US-15 and the Physical Solution Design;
- the merged Feature-001 specification and supporting artifacts;
- my confirmed Feature-002 design decisions recorded in the specification.

Do not reopen decisions that have already been explicitly resolved, including:

- US-14 owns concrete validation rules and acceptance criteria;
- Feature-002 / US-15 owns both the single-retry workflow and governed Unity
Catalog Volume-backed validated briefing storage;
- any non-ConfigurationError GenerationProvider failure is retryable;
- ConfigurationError is non-retryable;
- validation-failure retry uses the original context plus failed criteria and
Validation Feedback where available;
- generation-failure retry uses the original context without fabricated
validation feedback;
- Attempt 2 is always final;
- terminal generation and validation failures remain distinguishable;
- no invalid briefing is stored and any previously valid briefing is preserved;
- no new advisor-visible retry indicator is required;
- unwired legacy providers remain out of scope;
- Feature-002 must be independently testable using controlled generation and
validation outcomes.

Do not ask about implementation or architectural choices that belong in
/speckit.plan, such as exact modules/classes, dependency-injection wiring,
retry-prompt implementation, context-copy mechanics, Unity Catalog Volume
paths/file formats, configuration names, test-file placement, logging helpers,
or sync/async choices.

Ask only questions requiring a genuine behavioural, product,
acceptance-criteria, scope, or workflow decision from me.

Do not invent answers.
```

```
If no material ambiguity remains, state that clarification is complete rather
than manufacturing additional questions.

Integrate any confirmed clarification answers back into the Feature-002
specification.

Do not modify implementation code or perform any Git/GitHub write operations.
```

P17 - Generate Feature-002 implementation plan

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit.plan

Create the implementation plan for Feature-002: Briefing Retry and Validated
Briefing Storage, using the approved and clarified Feature-002 specification,
project constitution v1.1.0, and existing Feature-001 architecture.

Inspect the existing repository before determining implementation structure.

Use the Feature-002 repository investigation, approved specification and
Feature-001 artifacts as context. Treat the specification as authoritative for
required behaviour.

PRIMARY ARCHITECTURAL REQUIREMENT

Reuse and extend the architecture established by Feature-001 wherever reasonably
possible.

Feature-001 deliberately established the seams required by Feature-002,
including retry orchestration, generation, validation, briefing storage,
StudentService persistence orchestration, error handling and application
integration.

Do not create parallel implementations or redesign these boundaries where the
existing architecture can support Feature-002.

PLANNING REQUIREMENTS

1. Determine the minimum implementation necessary to satisfy the approved
Feature-002 specification.

2. Identify the exact existing files/components that should:
- remain read-only;
- be modified;
- be newly created only where genuinely required.

3. Define the concrete implementation of the single-retry workflow using the
existing generation, validation and retry boundaries.

4. Define how the retry request is constructed:
- validation failure uses the original generation context plus failed
acceptance criteria and Validation Feedback where available;
- generation failure uses the original context without fabricated validation
information.

5. Ensure Attempt 2 is always final and preserve the approved distinction
between terminal generation failure and terminal validation failure.

6. Define the concrete governed Unity Catalog Volume-backed implementation of
the existing BriefingStore abstraction while preserving Feature-001
persistence orchestration.
```

```
7. Determine the planning-level choices deliberately deferred by the
specification, including where necessary:
- module/class structure;
- dependency-injection/composition wiring;
- retry-prompt composition mechanism;
- context propagation/copy mechanics;
- Unity Catalog Volume path, storage format and naming approach;
- configuration requirements;
- attempt_count propagation;
- logging/observability details;
- test placement and test doubles;
- sync/async choices.

8. Preserve the existing behaviour that only validated briefings enter the
validated storage path and that previously stored valid briefings are not
removed by a failed retry/regeneration.

9. Keep the final US-13 generation implementation and US-14 validation rules
replaceable through the existing boundaries. Do not invent missing US-14
acceptance criteria.

10. Leave the currently unwired legacy briefing providers unchanged unless an
actual implementation dependency requires otherwise.

11. Define proportionate tests covering the approved Feature-002 specification,
including:
- validation failure then retry success;
- generation failure then retry success;
- Attempt 2 generation failure;
- Attempt 2 validation failure;
- no third attempt;
- correct use of validation feedback where available;
- invalid briefing not persisted;
- previous valid briefing preserved;
- Unity Catalog Volume-backed storage behaviour and storage failures.

12. Prefer the minimum necessary change. Do not include speculative refactoring,
unrelated cleanup, new frameworks, parallel architecture or infrastructure
outside Feature-002 scope.

Treat implementation choices deliberately deferred from the specification as
planning decisions.

If a technical decision would materially change approved behaviour or Feature-002
scope, flag it for human approval rather than silently changing the
specification.

Do not modify implementation code during planning.

Do not perform any Git/GitHub write operations.
```

P18 - Generate Feature-002 tasks

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit.tasks

Generate the actionable implementation tasks for Feature-002 from the approved
Feature-002 specification and implementation plan.

Requirements:
- Preserve the exact scope and implementation structure approved in the plan.
- Use the exact files, modules, classes and interfaces identified by the plan.
```

```
- Reuse the existing Feature-001 architecture and seams rather than introducing
parallel implementations.
- Do not introduce additional architecture, abstractions, dependencies or
functionality beyond what the approved plan requires.
- Order tasks by dependency.
- Make tasks small enough to implement and verify independently where practical.
- Clearly identify tasks that can safely be completed in parallel.

Include tasks necessary to implement:
- SingleRetryWorkflow;
- retry handling after validation failure;
- retry handling after non-ConfigurationError generation failure;
- retry request construction using original context plus failed criteria and
Validation Feedback where available;
- Attempt 2 validation and terminal generation/validation failure handling;
- enforcement of exactly one retry and no third attempt;
- attempt_count = 2 on successful retry;
- VolumeBriefingStore using the approved Unity Catalog Volume design;
- append-only validated briefing persistence and most-recent retrieval;
- storage configuration and composition-root wiring;
- preservation of previously stored valid briefings after failed retry/regeneration;
- required error handling and observability.

Pair implementation tasks with proportionate tests covering:
- validation failure → retry success;
- generation failure → retry success;
- Attempt 2 generation failure;
- Attempt 2 validation failure;
- no third generation attempt;
- validation feedback/failed criteria are used only when available;
- invalid briefings are never persisted as validated output;
- previous valid briefings are preserved;
- VolumeBriefingStore behavioural parity with the existing BriefingStore contract;
- storage failures are surfaced through the approved error boundary.

Preserve the five approved planning decisions:
- configurable BRIEFING_VOLUME path;
- unconditional SingleRetryWorkflow wiring;
- minimal Feature-002 retry-feedback wrapper;
- append-only briefing history with no pruning;
- one JSON file per validated briefing.

Do not create tasks for:
- US-13 OpenAI implementation;
- US-14 validation-rule implementation;
- unrelated legacy-provider cleanup;
- dashboard/frontend changes;
- unrelated refactoring or repository-wide cleanup;
- changes to the data-generation or machine-learning notebooks unless explicitly
authorised by the approved plan.

The resulting tasks must implement the approved Feature-002 plan, not reinterpret
or expand it.

Do not perform any Git/GitHub write operations.
```

P19 - Analyse Feature-002 before implementation

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit.analyze

Analyze Feature-002 for consistency across the approved constitution,
```

```
specification, clarification decisions, final plan, supporting planning
artifacts, approved planning decisions, and generated tasks.

Feature-002 is strictly bounded to Product Backlog US-15:
Briefing Retry and Validated Briefing Storage.

Check for:
- requirements without corresponding implementation tasks;
- tasks without corresponding approved requirements;
- contradictions between spec.md, plan.md, research.md, data-model.md,
contracts, quickstart.md, and tasks.md;
- inconsistencies with the approved Feature-002 design and planning decisions;
- scope leakage into US-13, US-14, frontend/dashboard work, or unrelated
application functionality;
- missing acceptance-criteria or success-criteria coverage;
- incomplete coverage of retryable generation failure, validation failure,
Attempt 2 outcomes, no-third-attempt behaviour, and persistence protection;
- incomplete coverage of the governed Unity Catalog Volume BriefingStore
requirements;
- conflicts with the existing Feature-001 architectural seams or persistence
orchestration;
- unnecessary implementation complexity, new abstractions, dependencies,
refactoring, or infrastructure;
- constitution violations.

Verify that the five approved planning decisions are represented consistently:
- configurable BRIEFING_VOLUME path;
- unconditional SingleRetryWorkflow wiring;
- minimal Feature-002 retry-feedback wrapper;
- append-only briefing history with no pruning;
- one JSON file per validated briefing.

Do not reopen previously approved product or planning decisions unless the
artifacts contain a direct contradiction that would prevent correct
implementation.

Do not modify implementation code.

If issues are found, identify:
- severity;
- the exact artifact(s) involved;
- the conflicting or missing requirement/task;
- the minimum correction required before implementation.

Do not perform any Git/GitHub write operations.
```

P20 - Implement Feature-002

Scope: Feature-002
Human review: Reviewed and approved by **Renny Matis** as part of the documented workflow.

Prompt Used (verbatim)

```
/speckit.implement

Implement Feature-002: Briefing Retry and Validated Briefing Storage using the
final approved constitution, specification, clarification decisions,
implementation plan, supporting planning artifacts, analysis corrections, and
tasks in specs/002-briefing-retry-and-storage/.

Treat the approved tasks and the exact files identified by the approved plan as
the implementation boundary.

Requirements:

- Follow the approved specification, plan, contracts, planning decisions, and
```

```
tasks exactly.
- Work through tasks in their approved dependency order.
- Implement only the minimum code necessary to satisfy each approved task.
- Modify or create only files explicitly authorised by the approved plan/tasks.
- Files identified as read-only, reference-only, outside Feature-002 scope, or
owned by other contributors must remain unchanged.
- Preserve all existing Feature-001 behaviour and architectural seams unless an
approved Feature-002 task explicitly requires a change.
- Reuse and extend the existing retry, generation, validation, persistence,
configuration, error-handling, and composition architecture rather than
creating parallel implementations.
- Preserve the approved Feature-002 / US-15 scope: single-retry workflow plus
governed Unity Catalog Volume-backed validated briefing storage.
- Do not implement US-13 generation-provider functionality, US-14 validation
rules, frontend/dashboard changes, legacy-provider cleanup, or unrelated
application functionality.
- Do not reinterpret, expand, or add requirements during implementation.
- Do not perform unrelated refactoring, cleanup, optimisation, dependency
upgrades, architectural redesign, or speculative improvements.
- Avoid repository-wide formatting or automated fixes that could modify files
outside the approved scope.

If implementation appears to require modifying a file not authorised by the
approved plan/tasks, STOP before changing it and report:
1. the file;
2. why the change appears necessary;
3. the requirement or dependency causing it;
4. the minimum proposed change;
5. whether existing team functionality could be affected.

Do not alter the approved specification, plan, or tasks simply to make them
match the implementation. Surface any inconsistency instead.

Complete the tests required by the approved tasks, including the final
post-analysis task corrections.

Run the Ruff, pytest, and other validation gates specified by the approved
tasks/quickstart, without automatic fix options that could alter unrelated
files.

Before making changes:
1. inspect git status;
2. inspect the approved Feature-002 task/file scope;
3. identify the files expected to be modified or created;
4. confirm that no unrelated or team-owned files require modification.

After implementation:
- report all files created or modified;
- report completed task IDs;
- report tests and validation commands run and their results;
- report any approved files/tasks that could not be completed;
- report any files considered for modification but left unchanged because they
were outside scope;
- report any unresolved issues requiring human review.

Do not commit, push, merge, rebase, switch branches, create pull requests, amend
commits, or otherwise perform Git/GitHub write operations.
```

Appendix B - Evidence Scope

This condensed worklog is derived from the original **Software Development Prompts - US-008 and US-014** worklog. The original document remains the full raw record of prompts and AI outputs. This version intentionally:

- preserves the prompts themselves in Appendix A;

- condenses long AI-generated repository investigations and analysis reports;
- records the resulting artefacts, implementation outputs and test results;
- makes human review and approval by **Renny Matis** explicit;
- is formatted to remain readable as both Markdown and PDF.